Vorlesung Algorithmen und Datenstrukturen

SS 2026

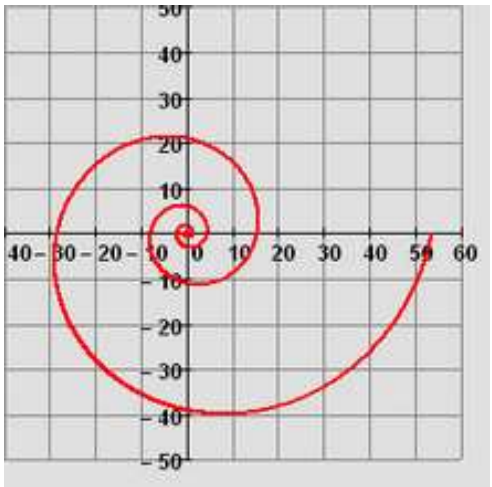
Prof. Dr. U. Göhner

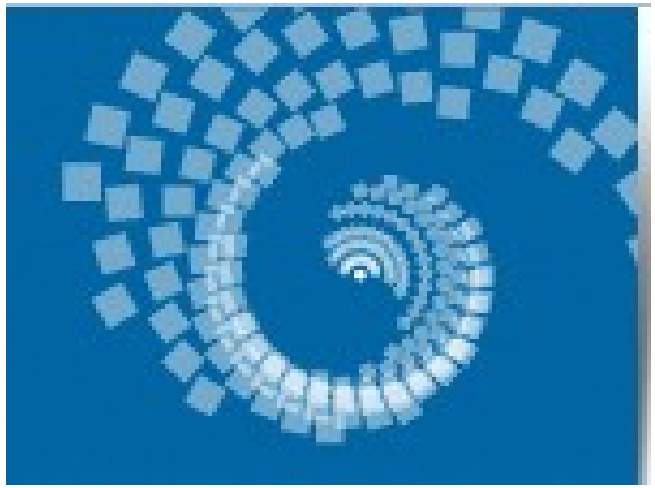
Hochschule Kempten

Ziele der Vorlesung

Theoretische Kenntnisse:

- Was zeichnet „effiziente“ Verfahren aus, und welche Algorithmen und Datenstrukturen stecken dahinter?
- Wie ist der Aufwand (CPU-Zeit, Speicherplatz) eines Verfahrens?
- Wie kann man diesen Aufwand in Komplexitätsmaße fassen und wie analysiere ich einen Algorithmus?





Ziele der Vorlesung

Praktische Fähigkeiten:

- Wie analysiere ich eine Aufgabenstellung,
- Wie bilde ich dieses dann auf ein Verfahren

  = Datenstruktur + Algorithmus ab?
- Wie implementiere ich ein lauffähiges Programm aus dem erarbeiteten Verfahren
- Wie teste ich es das Programm (Korrektheit)
- Wie teste ich Performance und Speicherbedarf

Ziele der Vorlesung

Warum ist das wichtig?

- Verarbeitung von großen Datenmengen (Big Data)
- Training von KI-Modellen
- LLM Beispiel Deepseek
  - Effizientere Algorithmen statt Compute Power
  - Verfeinerung der Entscheidungsmodelle

27 JANUAR 2025

DeepSeek: Wendepunkt für die KI-Branche?

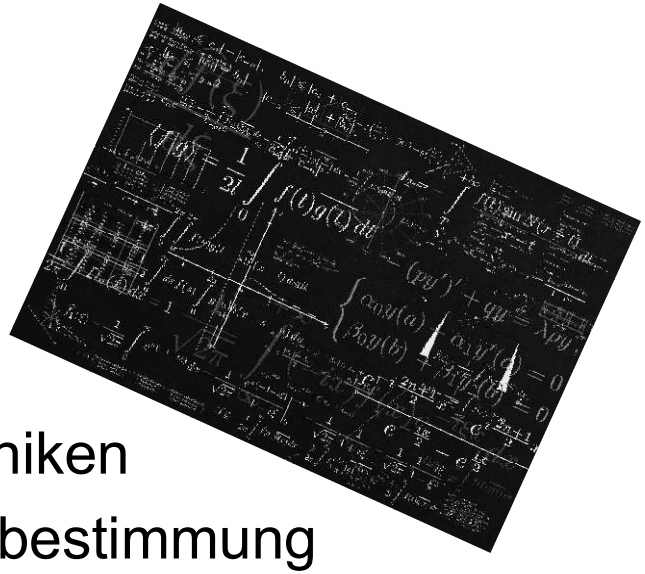
MÜNCHEN – DeepSeek, ein chinesisches Unternehmen für künstliche Intelligenz (KI), hat ein Open-Source-Modell lanciert, das relativ kostengünstig zu trainieren sein soll und weniger Energie und Rechenleistung benötigt als die bisherigen Anwendungen, wie Dominic Rizzo von T. Rowe Price, beschreibt. Als Reaktion darauf rauschten US-Tech-Aktien in den Keller. Ein Ausblick.

Allgemeines

Voraussetzungen:

- LV Programmieren, objektorientiertes Programmieren in C++

- Mathematik
  - Logik
  - Beweisführung
  - vollständige Induktion
  - allgemeine mathematische Techniken
    - Limes-Begriff und Grenzwertbestimmung
    - Rechentechniken



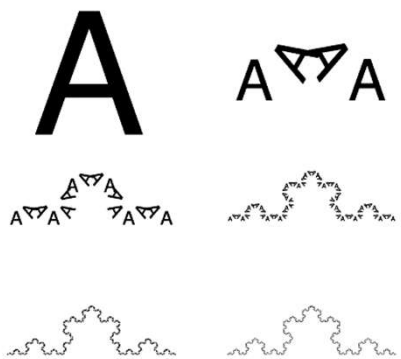
Allgemeines

- Prüfungsvorleistung (PVL)
  - aktive Übungsteilnahme:
    - Kontrolle über Votiersystem
      - >=70% votieren
      - >=3 Aufgaben vorführen
  - Erste Übung am 24.3.26/25.03.26/26.03.26
  - PVL wird nur im SS angeboten
- Prüfung nach dem SS
- AuD Voraussetzung für Vorlesungen SWT, DBI,....

Gliederung der Vorlesung

Einführung Algorithmus

- Definition Algorithmus
- Darstellungsarten für Algorithmus
- Komplexität und O-Notation
- Algorithmen-Schemata:
  - Greedy-Algorithmen
  - Rekursion
  - Divide and Conquer-Algorithmen
  - iterative Algorithmen



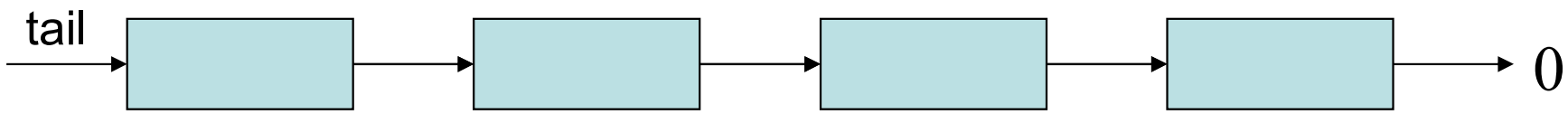
rekursiv erzeugtes
„Koch-Fraktal“

Niccolo Macchiavelli
„Divide et Impera“

Gliederung der Vorlesung

Einführung Datenstruktur

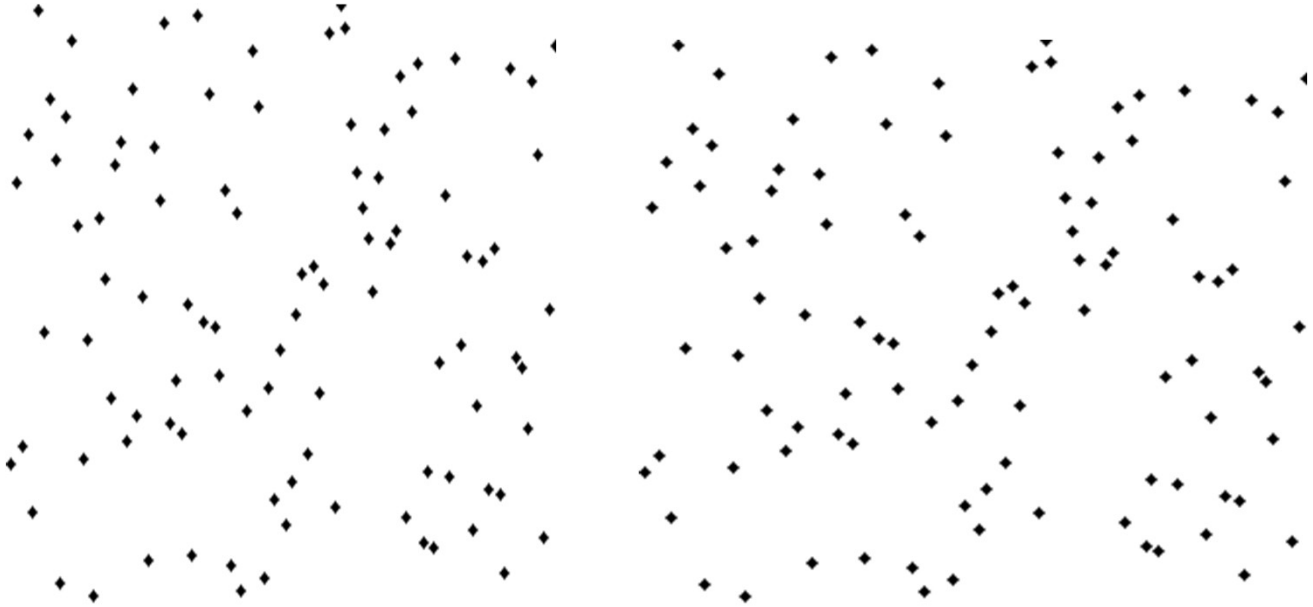
- Definition Datenstruktur / Abstrakter Datentyp
- einfache Datentypen
- dynamische Datenstrukturen
  - verkettete Liste
  - Stack und Queue
  - Tabelle
    - Zugriffsoperationen
    - Implementierungen



Gliederung der Vorlesung

Sortieralgorithmen

- elementare Sortierverfahren
  - Insertion-, Selection-, ExchangeSort
- schnelle Sortierverfahren
  - Quicksort, Mergesort, Heapsort, Shellsort, Combsort
- Aufwandsabschätzungen (Komplexitätsabschätzung)

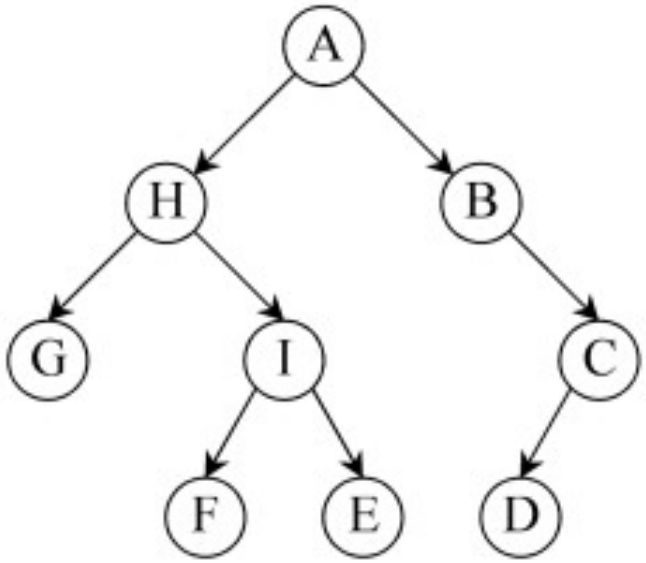


Wikimedia Commons

Gliederung der Vorlesung

Baumstrukturen

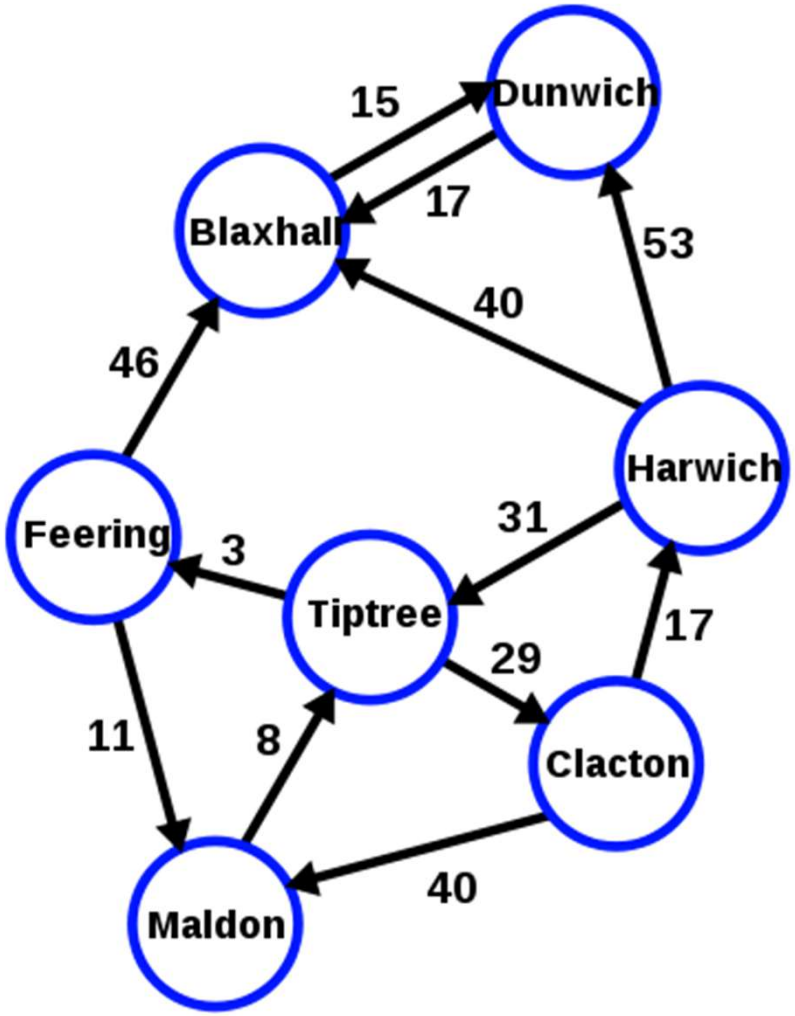
- Begriffsbildung
- Operationen auf Bäumen
- Suchbaum
- Heap, treap
- Ausgeglichene Bäume
- AVL-Eigenschaft
- B-Baum



Gliederung der Vorlesung

Hashfunktion
- Definition
- Anwendung

Graphen
- Definition
- Darstellung
- Implementierung
- Breiten/Tiefensuche
- Reflexive transitive Hülle
- Kürzeste Wege
- Längste Wege



Literatur

- Wirth: Algorithmen und Datenstrukturen (Pascal, Modula2, Oberon)
- Sedgewick: Algorithmen in C++
- Saake/Sattler: Algorithmen und Datenstrukturen (Java)
- Reeß/Viebeck: Datenstrukturen und Algorithmen (C++)
- Heun: Grundlegende Algorithmen (C)
- Logofatu: Algorithmen und Problemlösungen mit C++
- Pomberger/Dobler: Algorithmen und Datenstrukturen
- Edmonds: How to Think about Algorithms
- Gumm / Sommer Informatik, Band 1: Programmierung, Algorithmen und Datenstrukturen
- Cormen / Leiserson / Rivest / Stein: Algorithmen – Eine Einführung

-> z.T. auch als ebooks in Bibliothek verfügbar

Literatur

Niklaus Wirth - schweizer Informatiker:

- Grundlegende Arbeiten auf den Gebieten:
  - Algorithmen, Datenstrukturen
  - Programmiersprachen
  - Modulare strukturierte Programmierung

Wikimedia Commons

Zitate:

- “Programming is a constructive art.”
- “Reliable and transparent programs are usually not in the interest of the designer.”
- “In our profession, precision and perfection are not a dispensable luxury, but a simple necessity.”
- „Software wird schneller langsamer als Hardware schneller wird.”

Definition Algorithmus

Definition:

Ein Algorithmus ist eine eindeutige, endliche Beschreibung eines allgemeinen, endlichen Verfahrens zur schrittweisen Ermittlung gesuchter Größen aus gegebenen Größen

Beschreibung des Algorithmus:

Die Beschreibung erfolgt in einem Formalismus mit Hilfe von anderen Algorithmen. Ein Algorithmus muss für zulässige Eingaben ausführbar sein.

Algorithmus

Die Bezeichnung Algorithmus entstand aus dem Namen eines arabischen Mathematikers,

Muhammed idn Musa abu Djafar al-Choresmi,

auch *Al Khwarizmi*

oder al-Khowarizmi,

geboren um 780 südöstlich des Aral-Sees, in Choresmien im heutigen Usbekistan, gestorben um 850.

Al Khwarizmi-Skulptur in Khiva,
Wikimedia Commons

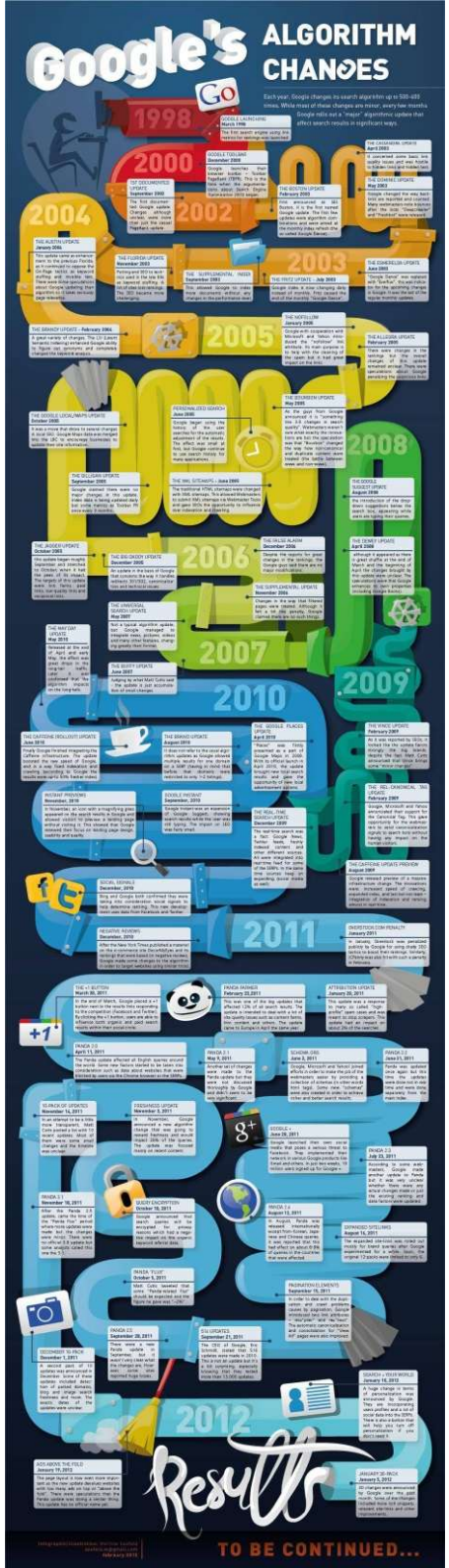
Guter Algorithmus:

Eigenschaften eines guten Algorithmus:

- schnell
- wenig Speicherbedarf
- einfach und leicht verständlich (->kleine Entwicklungskosten)

Beispiele:

- Suche von Datensätzen
  - Verzeichnis einer Bibliothek,
  - Kontenverzeichnis einer Bank
  - allgemeine Datenbanken
- Pfade in einem Graphen:
  - kürzester Pfad von A nach B,
  - kürzester Pfad, der alle Knoten miteinander verbindet
- Suche nach CharacterFolgen ("Mustern“) im Text (Suchmaschine)
- KI – Algorithmen
- Machine Learning

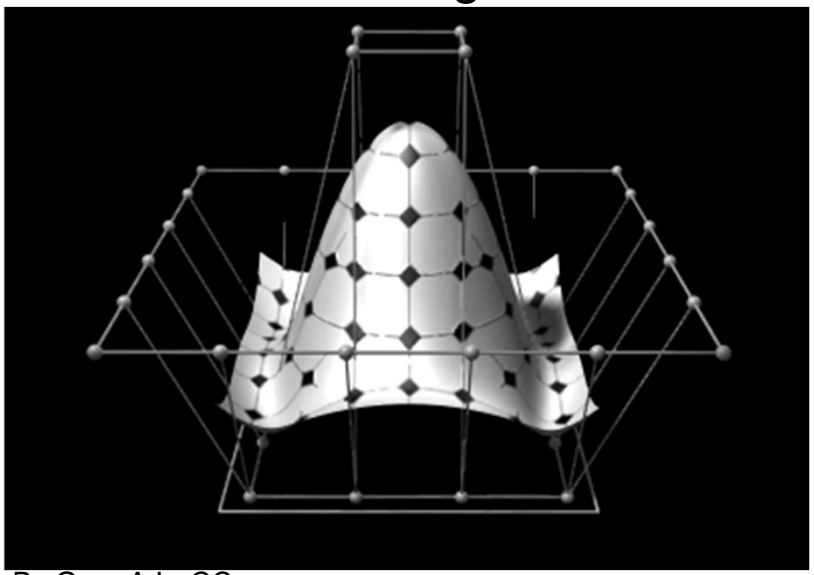


Beispiele:

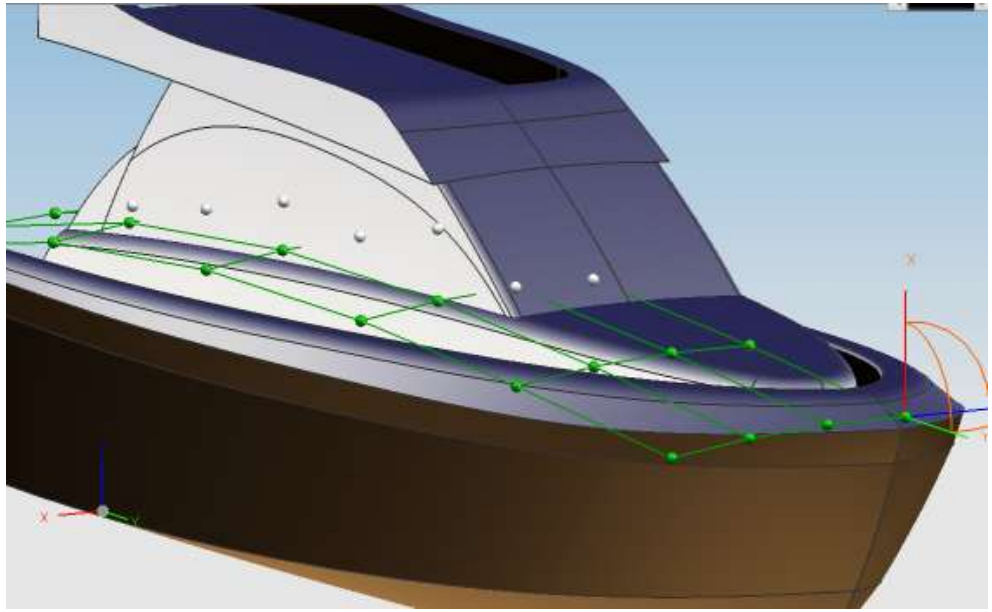
1) Auswertung von Polynomfunktionen:

Anwendungsbeispiele:

- Zeichenprogramme
- CAD-Systeme
- ContentCreation
- Bild/Mustererkennung
- Rendering



By Greg A L -CC



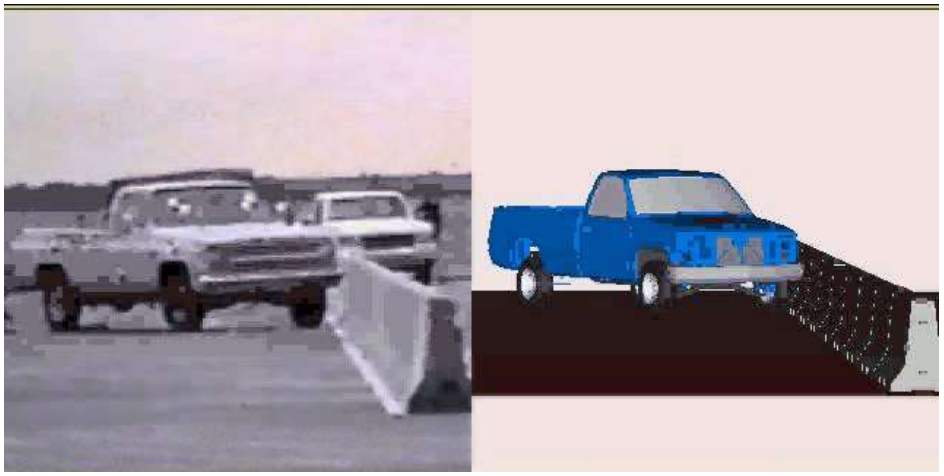
By Freeformer - CC

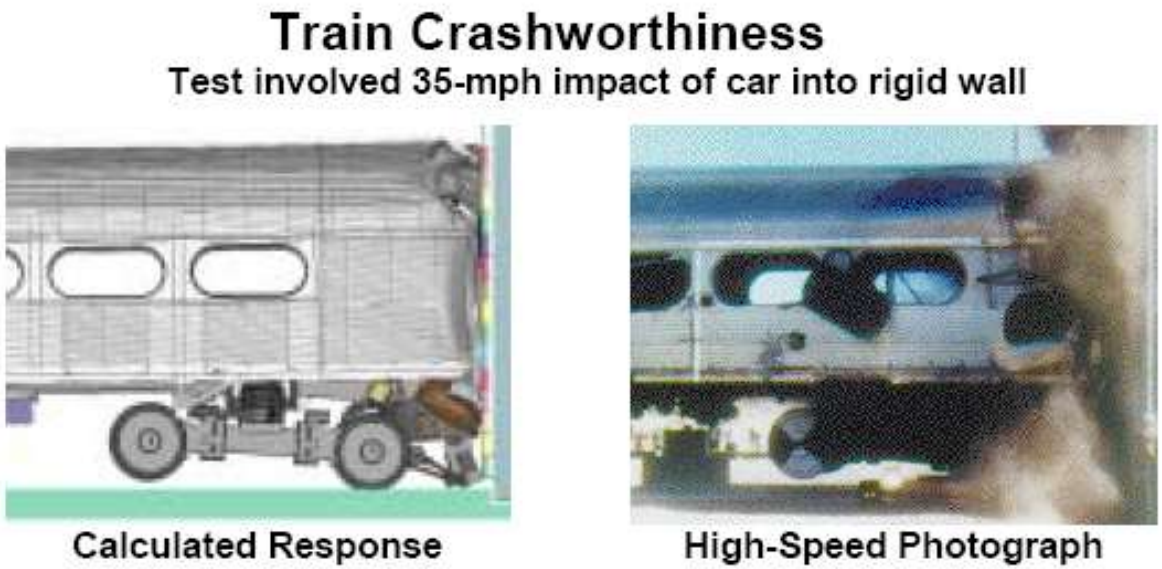
Beispiele:

2) Matrix-Vektor-Operationen

Anwendungsbeispiele:

- graphische DV (Rotation)
- Simulation
- Betriebswirtsschaftliche Optimierungen



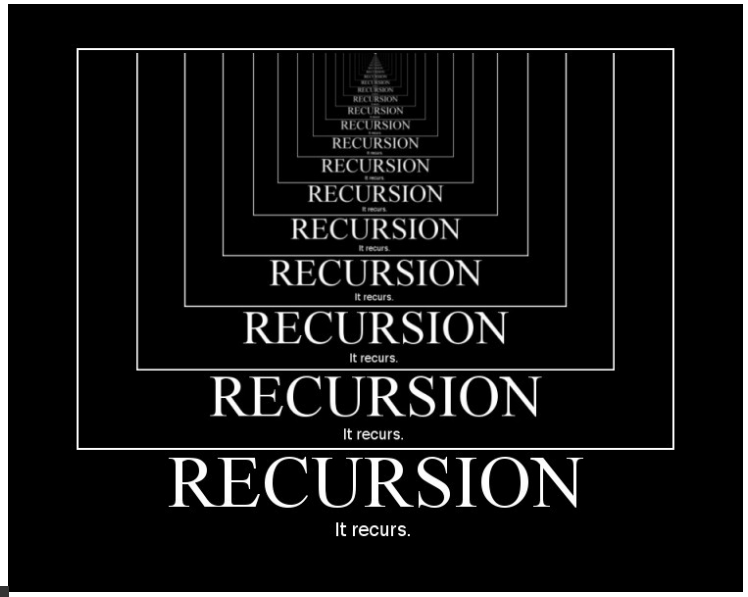


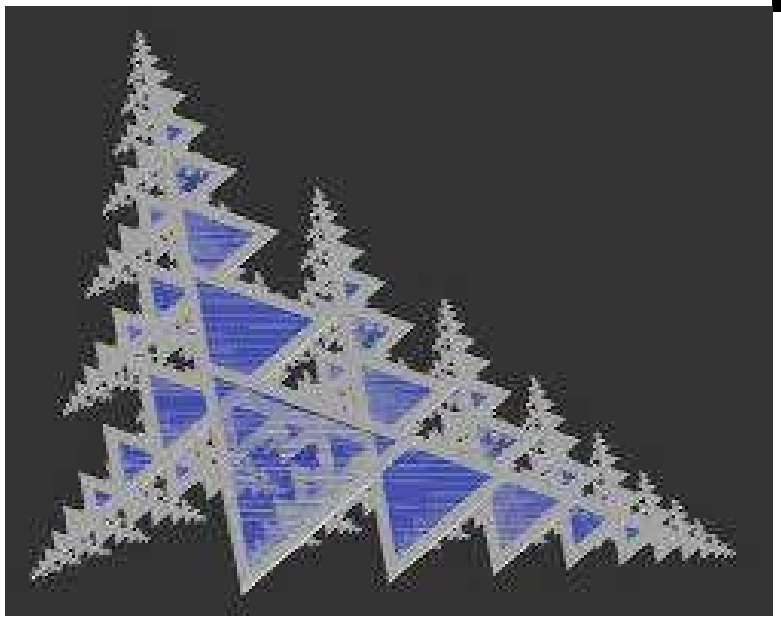
Beispiele:

3) Rekursion

Beispiele:

- Fakultätsfunktion
- Summenbildung
- Suchalgorithmen





„UM REKURSION ZU VERSSTEHEN, MUSS MAN ZUNÄCHST REKURSION VERSTEHEN“

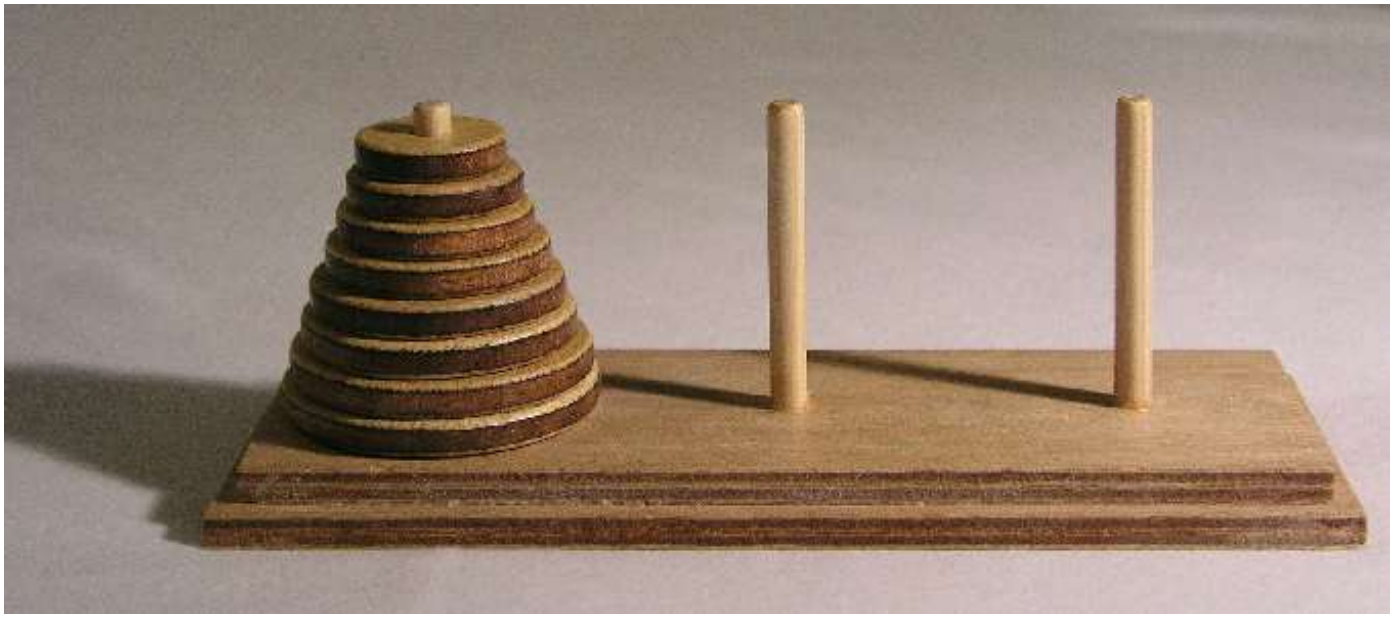
Beispiele:

3) Rekursive Algorithmen

Beispiel: Türme von Hanoi

Regeln des Spiels:

- es darf immer nur eine Scheibe bewegt werden
- es darf nur eine kleinere auf eine größere Scheibe gelegt werden.



Beispiele:

3) Rekursive Algorithmen

Beispiel: Türme von Hanoi

Falls Lösung für Turm mit n-1 Scheiben bekannt, kann auch die Aufgabe für einen Turm mit n Scheiben gelöst werden.

Rekursive Vorgehensweise:

1. Transportiere (rekursiv) Turm mit n-1 Scheiben von Start auf Zwischenablage
2. Transportiere verbliebene Scheibe von Start ins Ziel
3. Transportiere (rekursiv) Turm mit n-1 Scheiben von Zwischenablage ins Ziel.

Beispiele:

3) Rekursive Algorithmen

Beispiel: Türme von Hanoi

Beachte:

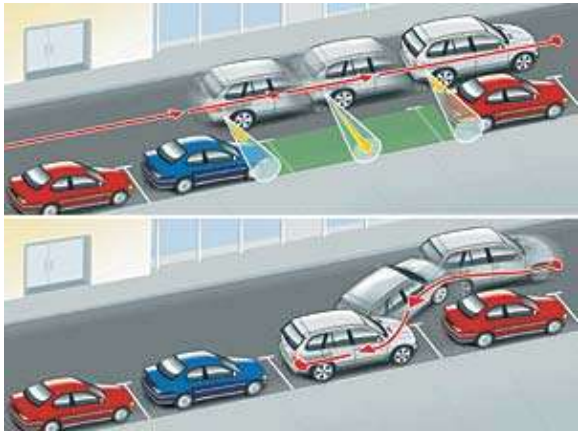
1. Die rekursive Lösung in Schritt 1 und 3 beachtet die Regeln des Spiels (keine größere Scheibe auf kleinerer Scheibe, immer nur 1 Scheibe transportieren)
2. Die Rekursion muss beendet werden, z.B. für Turm mit Höhe 1: triviale Lösung ausgeben.

Algorithmen in der Praxis

4) Wechselgeld zurückgeben beim Kaffee-Automat

5) Automobilindustrie:

- Objekterkennung / Spracherkennung
- Airbag-Auslöse-Algorithmen
- Erkennen von Hindernissen
- Erkennen von Fahrerschläfrigkeit
- Parkassistent
- autonomes Fahren



Folgerungen:

1. Algorithmen spielen eine wichtige Rolle im Alltag und innerhalb der technisierten Welt allgemein.
2. Effizienz der Algorithmen trotz schneller Rechner wichtiges Ziel.
3. Oft können nicht alle Ziele gleichzeitig erreicht werden, dann müssen Prioritäten gesetzt werden, z.B. Speicherbedarf.
4. Effektivität der Algorithmen hängen stark mit der zugrunde liegenden Datenstruktur zusammen.
5. Bei Verwendung von Rekursionen kann einfacher Code zu vielen Rechenoperationen führen.

Begriff: Terminierung

Definition terminierender Algorithmus:

Ein Algorithmus heißt terminierend, wenn er bei jeder erlaubten Eingabe nach endlich vielen Schritten abbricht.

Fragestellung:

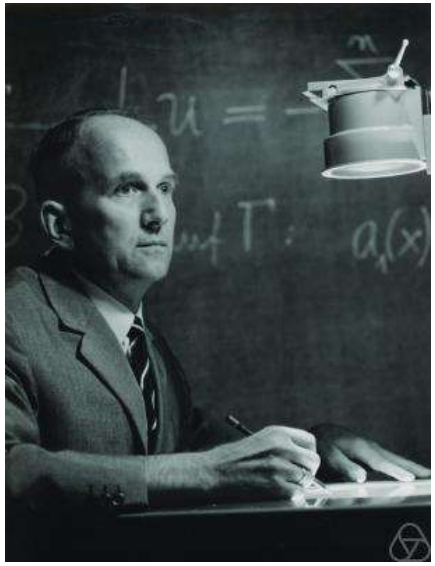
- Kommt der Algorithmus irgendwann zu einem Ergebnis oder rechnet er „ewig weiter“
- Wichtiges Problem der Theoretischen Informatik

HHGTTG.....42?

Begriff: Terminierung

Beispiel Collatz-Funktion:

$$c_{n+1} = \begin{cases} \frac{c_n}{2}, & falls\ c_n\ gerade \\ 3 * c_n + 1, & falls\ c_n\ ungerade \end{cases}$$



Lotha Collatz 1910-1990

Collatz-Vermutung:

- Die Collatz-Folge endet immer in dem Zyklus 4,2,1,…
- Die rekursive Funktion collatz terminiert mit Rekursionsende $c_n = 1$
- Bis heute weder bewiesen noch widerlegt

Collatz(19):
19, 58, 29, 88, 44, 22, 11, 34, 17, 52, 26, 13, 40, 20, 10, 5, 16, 8, 4, 2, 1, 4,2, 1,4,2,1,…..

Begriff: Terminierung

Halteproblem:

- Bedeutendes Problem in der theoretischen Informatik mit wichtigen Konsequenzen für die Praxis:

Halteproblem: „Gibt es ein Verfahren (Algorithmus!), mit dem man für jeden Algorithmus entscheiden kann, ob dieser terminiert?“

Halteproblem ist ein klassisches Beispiel (von vielen) für ein nicht entscheidbares/berechenbares Problem -> THI Teil2, Berechenbarkeit

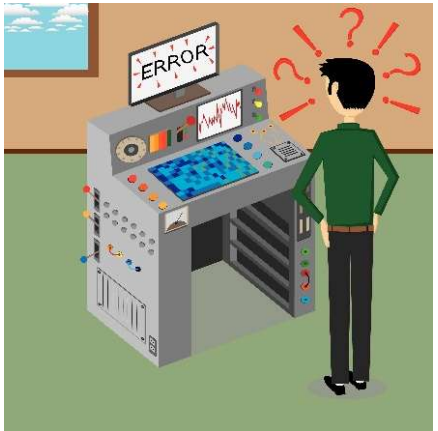
Begriff: Terminierung

Terminierung kann nachgewiesen werden für

- Spezielle Algorithmen
- Klassen von Algorithmen
- Spezielle Programstrukturen (z.B. Schleifen)
- Untersuchungen zur Terminierung von Programmen können im Rahmen der Verifizierung von Programmen durchgeführt werden

Aber es gibt kein allgemeingültiges Verfahren.

Für manche Probleme ist Terminierung **nicht** entscheidbar



Begriff: Determinismus

Definitionen:

1. Ein Algorithmus heißt deterministisch wenn er eine eindeutige Schrittfolge besitzt.
2. Ein Algorithmus liefert ein determiniertes Ergebnis, wenn bei vorgegebener Eingabe auch bei mehrfacher Durchführung stets ein eindeutiges Ergebnis berechnet wird.
3. Nichtdeterministische Algorithmen mit determiniertem Ergebnis heißen determinierter Algorithmus.

Begriff: Korrektheit

Definition:

1. Ein Algorithmus heißt korrekt, wenn er für alle zulässige Eingaben ein korrektes Ergebnis erzeugt.
2. Ein korrektes Ergebnis für zulässige Eingaben wird über eine Spezifikation definiert.

Verifikation der Korrektheit eines Algorithmus:

- theoretisch bewiesen oder
- über (ausreichend viele) Testfälle überprüft

Darstellungsformen

1. Verbale Darstellungsform

Beispiel Darstellung eines Algorithmus in verbaler Form:

- Auszugeben ist der Mittelwert von N einzulesenden positiven Zahlen
- Bei der Eingabe soll geprüft werden, ob die Zahlen positiv sind, Eingaben <= 0 sollen ignoriert werden.

Darstellungsformen

2. Pseudocode-Darstellung:
Beispiel Darstellung eines Algorithmus in Pseudocode:

```
read N
Zähler = 1
Summe = 0
while (Zähler <= N) do
  read Zahl
  if (Zahl > 0)
   then
     Summe = Summe + Zahl
     Zähler = Zähler + 1
  end-if
end-while
Schnitt = Summe/(Zähler-1)
write Schnitt
```

Darstellungsformen

2. PseudoCode Formen

Adele (**a**lgorithm **de**scription **l**anguag**e**)

- an Pascal und Modula2 angelehnt
- Rechenberg / Pomberger TU Berlin

Jana (**J**ava-Based **A**bstract **N**otation for **A**lgorithms)

- An Java angelehnt
- Johannes Kepler Universität Linz

Je nach Anwendung unterschiedliche Formen von PseudoCode:

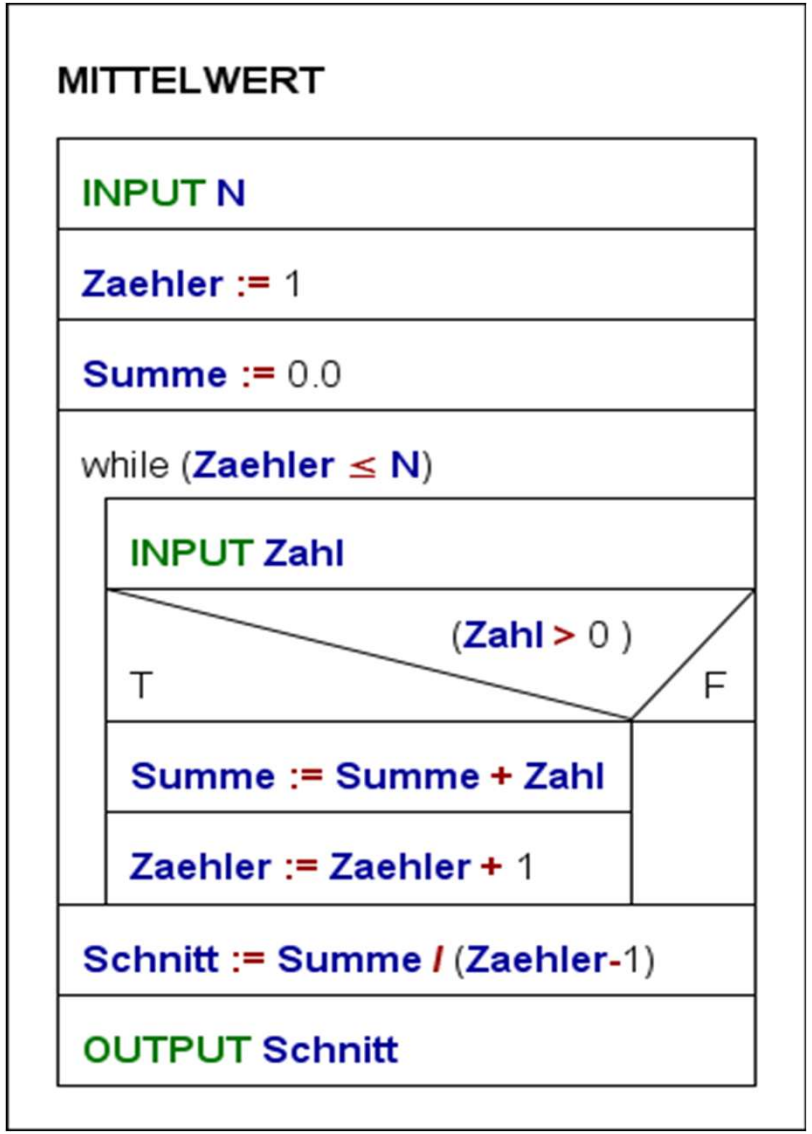
- z.B. bei DataScience / KI-Anwendungen ist eine Mischung aus Python und Mathematischen Beschreibungen üblich

Struktogramme von Nassi und Shneiderman (DIN 66261)

Tools für die Erstellung von Struktogrammen:

Structorizer:

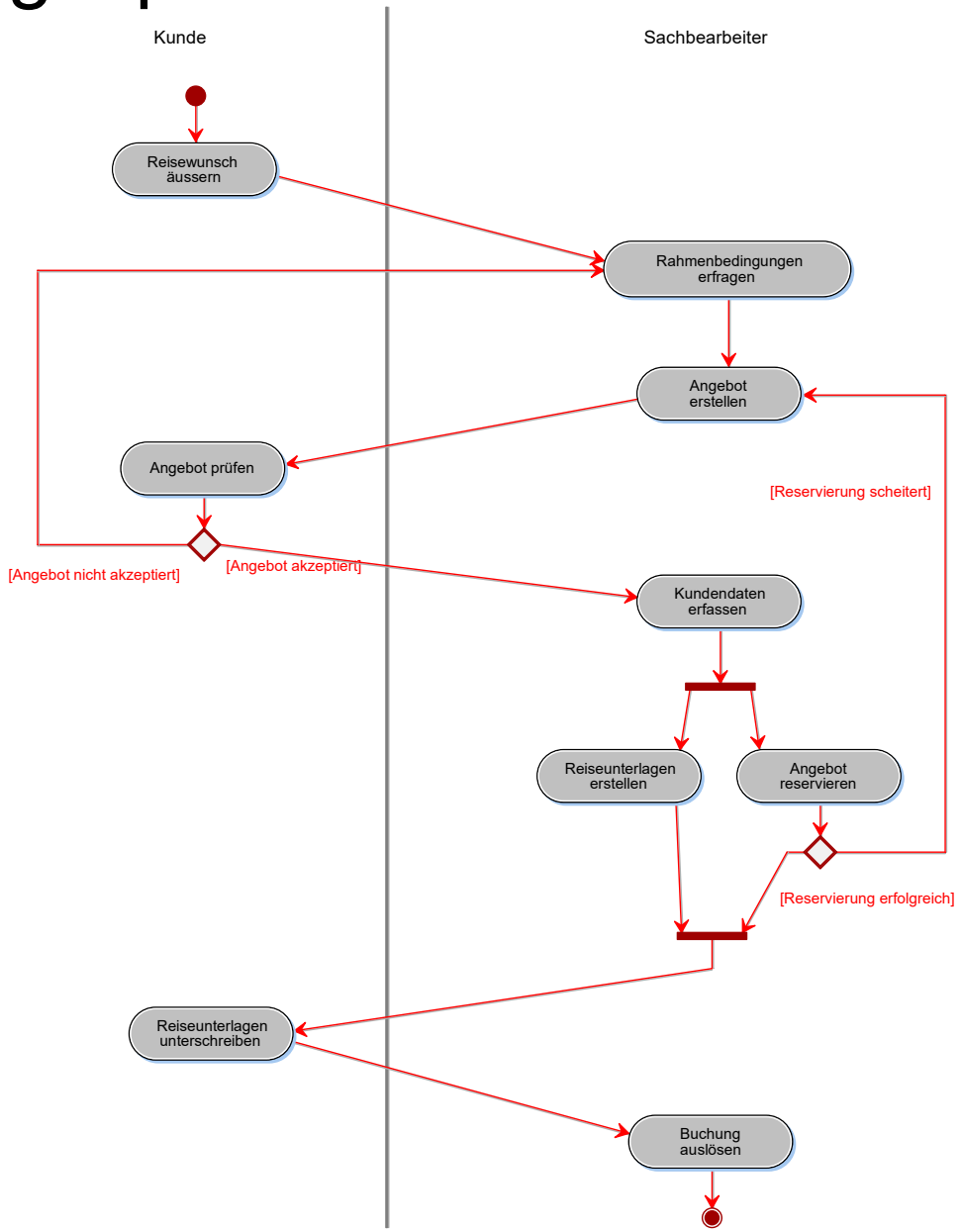
- auch Ausgabe von Quellcode möglich
- Test auf Korrektheit möglich
- Debug-optionen
- Darstellung Laufzeiten



UML = Unified Modelling Language

9 Modellierungssprachen

- Aktivitätsdiagramm
- Sequenzdiagramm
- Ausführlich behandelt in -> LV Softwaretechnik



Schnelligkeit eines Algorithmus

Rechenzeit eines Algorithmus ist abhängig von:

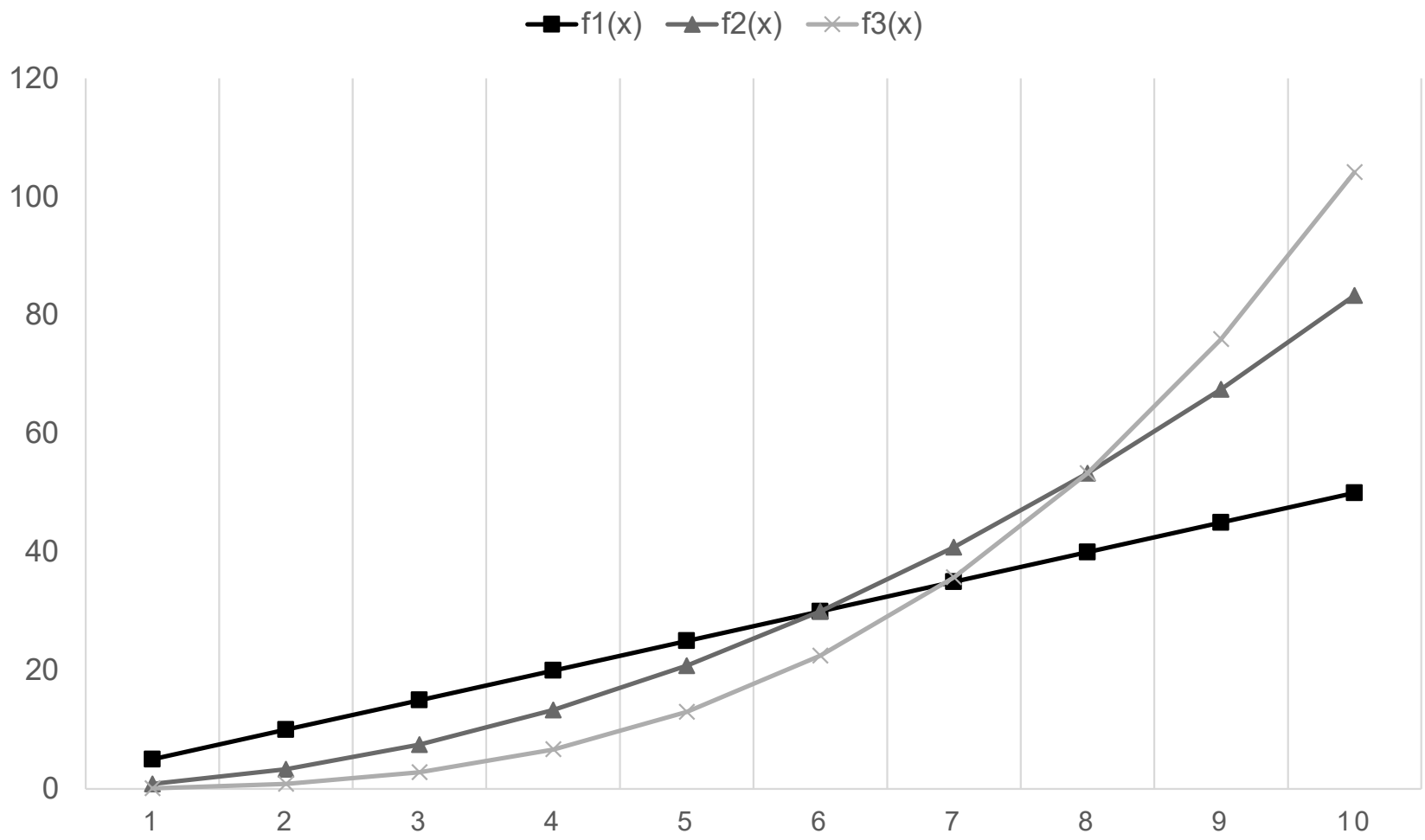
- den Eingabedaten,
- der Qualität des Compilers,
- der Hardware und dem Betriebssystem

Komplexitätsanalyse berücksichtigt nur die Abhängigkeit von den gewählten Eingabedaten:

- Größe der Eingabedaten
- Struktur der Eingabedaten

Komplexitätsanalyse von Algorithmen

Vergleich des asymptotischen Verhaltens für große Laufzeiten



Komplexität von Algorithmen

Big-O-Notation = Landausche Symbole

f(n) und g(n) seien positive monoton wachsende Funktionen,

dann gelten folgende Definitionen:

Definition:

O(g(n)) ist die Menge aller Funktionen f(n), die gleich schnell oder langsamer wachsen als g(n).

Es gibt ein c aus der Menge R^+ für die gilt:

$$\lim_{n \to \infty} \left(\frac{f(n)}{g(n)} \right) \leq c$$

Big-O-Notation

Definition:

$\Theta(g(n))$ ist die Menge aller Funktionen f(n), die gleich schnell wachsen wie g(n).

Es gibt ein c aus der Menge R^+ für die gilt:

$$\lim_{n \to \infty} \left(\frac{f(n)}{g(n)} \right) = c$$

Big-O-Notation

Definition:

$\Omega(g(n))$ ist die Menge aller Funktionen f(n), die gleich schnell oder schneller wachsen als g(n).

Es gibt ein c aus der Menge R^+ für die gilt:

$$\lim_{n\to\infty}\left(\frac{f(n)}{g(n)}\right) \geq c$$

oder $\frac{f(n)}{g(n)}$ wächst unbeschränkt für $n\to\infty$

Big-O-Notation

Definition:

o(g(n)) ist die Menge aller Funktionen f(n), die langsamer wachsen als g(n).

$$\lim_{n \to \infty} \left(\frac{f(n)}{g(n)} \right) = 0$$

Komplexitätsklassen

1. $O(1)$ konstant	elementarer Befehl
2. $O(\log(\log n))$	doppelt logarithmisch
3. $O(\log n)$ logarithmisch	Binärsuche
4. $O(n)$ linear	lineare Suche
5. $O(n \log n)$ überlinear	schnelle Sortierverfahren
6. $O(n^2)$ quadratisch	einfache Sortierverfahren
7. $O(n^3)$ kubisch	Matrizen-Inversion
8. $O(n^k)$ polynomiell	vom Grad k lineare Programmierung
9. $O(2^n)$ exponentiell	Turm von Hanoi
10. $O(n!)$ Fakultät	Traveling Salesman Problem
11. $O(n^n)$	

Laufzeiten

⟶ Nur Algorithmen bis zu max. polynomialer Ordnung praktikabel

Komplexität	1 Sekunde	Zeit für n=1000000
n	1000000	1 Sekunde
n log n	100000	4 Sekunden
n^2	1000	12 Tage
n^3	100	31710 Jahre
2^n	20	10^{300000} Jahre

Beispiel Lineares Suchen in Listen

Lineare Suche in einem unsortierten Array

```
lin_suche(item_type suchwert)
{
        int l_u = 0, l_o = dimension-1;
        int suchindex = -1;
        for (int i=l_u; i<=l_o; i++)
                if (daten[i] == suchwert)
                        return i; // suchindex
        }
        return suchindex;
}
```

Beispiel Lineares Suchen in Listen

Lineare Suche in einem unsortierten Array

Falls Element y in der Liste vorhanden,

Fallunterscheidung:

1.: Im günstigsten Fall: **1 Vergleich**

2.: Im ungünstigsten Fall: **n Vergleiche**

3.: Im mittleren Fall: **(n+1)/2 Vergleiche**

P und NP

➔Theoretische Informatik

Die Klasse P enthält alle Entscheidungsprobleme, die in polynomieller Zeit gelöst werden können, d.h. es gibt einen Algorithmus gibt, der ein Ergebnis in der Laufzeit $O(n^k)$ liefert.

P und NP

➔Theoretische Informatik

Die Klasse NP enthält alle Entscheidungsprobleme, für die eine gegebene Lösung in polynomieller Zeit überprüft werden kann.

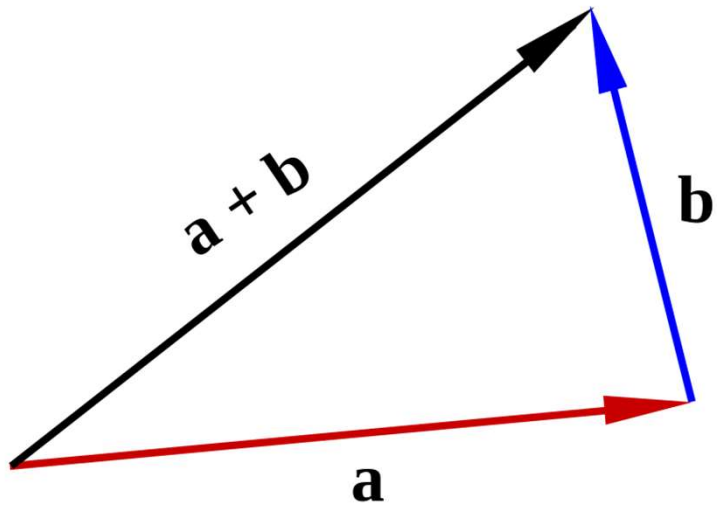
Algorithmen-Schemata

- Iterative Algorithmen
  - fixe Anzahl von Schleifendurchläufen
- Greedy - Algorithmen (der ‚gierige‘)
  - suche nach lokalem Optimum (z.B. steilster Abstieg)
- Rekursion
  - Funktion, die sich selbst aufruft
  - kann in iterative Prozedur umgewandelt werden
- Divide and Conquer-Algorithmen
  - Zerlegung in Teilprobleme
  - Zusammensetzung der Teillösungen zu einer Gesamtlösung

Iterative Algorithmen

Beispiel Vektor-Addition:

```
void add ( int n, *u, *v, *r)
{
    for (i = 0; i <n; i++)
            r[i] = u[i]+ v[i];
}
```



Greedy - Algorithmen

- Einfacher und schneller Algorithmus zur Lösung von Optimierungsproblemen:
  - Bestimmt in jedem Schritt die lokal optimale Lösung
  - „Nimm immer das größte zuerst“ (greedy=gierig)
- Vorteile Greedy:
  - nur polynomiale Komplexität
  - alle möglichen Pfade absuchen: exponentielle Komplexität
  - Lokales Optimum oft ausreichend (Verbesserungspotenzial)
- Nachteile Greedy
  - globales Optimum wird i.a. nicht gefunden

Greedy - Algorithmen

- Beispiele:
  - Wechselgeldalgorithmus
  - Scheduling von Prozessen auf Prozessoren
  - Minimalpfad in einem Graphen (Routenoptimierung)

Rekursion

- Funktion ruft sich selbst auf
- Terminationsbedingung verhindert Endlos-Schleife

```
int fact( int n)
{
   if (n<=1)
            return 1;                    // Terminationsbedingung
    else
            return n*fact (--n);         // rekursiver Aufruf
}
```

Rekursion

- Anzahl der rekursiven Funktionsaufrufe muß überschaubar bleiben (Speicherverbrauch)
- Algorithmus soll durch Rekursion einfacher formulierbar werden
- Komplexität mit Rekursion soll von derselben Ordnung sein wie ohne Rekursion

Beispiele Rekursion

Fibonacci-Zahlen

```
fibonacci(1) = 1
fibonacci(2) = 1
fibonacci(n+2) = fibonacci(n) + fibonacci(n+1)

long fibonacci( int n)
{
  if (n<=2)
   return 1;
  else
    return fibonacci(n-1) + fibonacci(n-2);
}
```

Rekursionsauflösung

- Umdrehen der Berechnung (von unten nach oben)
- Abspeichern der Zwischenresultate auf stack oder array

```
int fibo_iterativ(int n)
{
    int* afib;
    afib = new int[n+1];
    afib[1] = 1;
    afib[2] = 1;
    for (int i = 3; i <= n; i++)
        afib[i] = afib[i - 1] + afib[i - 2];
    return afib[n];
}
```

Rekursionstiefe

- Rekursive Form:
  - Anzahl der Vergleiche=Anzahl der rekursiven Aufrufe:
    - $T(n)= 1$ falls $n<=2$ oder
    - $T(n-1)+T(n-2)+1$, falls $n>=3$
  - n Speicherplätze im Stack nötig.
- Aufgelöste Form:
  - n+1 Speicherplätze nötig
  - n-1 Schleifendurchläufe mit jeweils
    - 1 Additionen + 1 Vergleich + 1 Inkrement

Rekursion

- dynamisches Programmieren Fibonacci-Zahlen:
  - rekursive Form, die Neuberechung von bekannten Werten vermeidet
  - O(n) Vergleiche

```
int fibo_dynamic(int n)
{
    static int known_fib[100];
    known_fib[1] = 1;
    known_fib[2] = 1;
    int result;
    if (known_fib[n] != 0)
        result = known_fib[n]; // Ergebnis bereits bekannt
        else
        {
            result = fibo_dynamic(n - 1) + fibo_dynamic(n - 2);
            known_fib[n] = result;
        }
    return result;
}
```

Rekursion

Weitere Beispiele

- Rucksackproblem (dynamisches Programmieren)
  - Dieb raubt Safe aus
  - Safe enthält N Typen von Gegenständen
  - Gegenstände haben unterschiedlichen Wert und Größe
  - Rucksack hat bestimmtes Fassungsvermögen cap
  - Gesamtwert soll maximiert werden

Greedy-Algorithmus für Rucksack-Problem

- Ordne Wertstücke der Reihenfolge nach dem Quotienten aus Wert zu Größe („möglichst leichte wertvolle Stücke“)
- Ergibt lokales aber nicht globales Optimum
- Geht aber schnell!

700 g
700 €

1000 g
1100 €

200 g
100 €

Rekursion

Rucksackproblem rekursive Lösung:

- Durchsucht alle Möglichkeiten
- Wiederholte Berechnungen werden ausgeschlossen (dynamisches Programmieren)
- Globales Optimum wird gefunden

```
int knap(int cap, int* itk)
{
   int i, space, max, t;
   for (i=0,max=0;i<N;i++)
    if ((space = cap-items[i].size) >=0)
     if ((t=knap(space, itk) +items[i].val) > max)
      {
        max=t; // neues Wert-Maximum
        itk[cap]=i; // momentan gepacktes Wertstück
      };
   return max;
}
```

DAQ - Divide and Conquer

Divide et Impera - Teile und Herrsche

1. Zerlegung in kleinere Teilprobleme (divide)
2. Löse Teilprobleme (conquer)
3. Setze Teile zur Gesamtlösung zusammen (merge)

Beispiele:

- Binäre Suche
- Mergesort, Quicksort,…

Binäre Suche in einem sortierten Array

Binäre Suche:

- ist ein Beispiel für Anwendung der Divide and Conquer-Strategie
- gegeben sortiertes Array a, Suchwert sw
- gesucht Suchindex si mit a[si]=sw
- Vorgehensweise:
  - funktioniert nur in sortiertem array
  - Sucharray ist Ausgangsarray
  - Solange Sucharray größer als ein Eintrag ist:
    - Aufteilung des Sucharrays in 2 Teilfelder
    - Falls sw im rechten Teilfeld setze Sucharray auf das rechte Teilfeld, sonst auf das linke Teilfeld
  - Rückgabe des Index (=si) des einzigen Elements im Sucharray

Suche in einem sortierten Array

Lineare Suche:

1. Im günstigsten Fall: 1 Vergleich
2. Im ungünstigsten Fall: n Vergleiche
3. Im mittleren Fall: (n+1)/2 Vergleiche

Binäre Suche:

1. Im günstigsten Fall: 1 Vergleich
2. Im ungünstigsten Fall: $\log_2(n)$ Vergleiche
3. Im mittleren Fall: $\frac{1}{2}(1+\log_2(n))$ Vergleiche